



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“SEMANA 2: Arreglos - Fundamento de las Estructuras”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

U. A. ESTRUCTURA DE BASES DE DATOS.....	1
COMPRENDE.....	3
1 Visualizador de Memoria.....	3
2 Detective del Caso MegaStore.....	4
Checkpoint Metacognitivo - Fase COMPRENDE.....	4
3 Arquitecto de Código.....	5
4 Laboratorio de Problemas Clásicos.....	7
Checkpoint Metacognitivo - Fase APLICA.....	10
5 Diálogo Socrático sobre Decisiones de Diseño.....	10
6 Identificación de Patrones en Problemas de Arreglos.....	12
Checkpoint Metacognitivo - Fase REFLEXIONA.....	14
7 Cazador de Bugs.....	14

COMPRENDE

1 Visualizador de Memoria

Diagrama propio que muestre la fórmula de acceso aplicada a un ejemplo diferente

Dirección	Valor	Índice
5000	1.1	arr[0]
5008	2.2	arr[1]
5016	3.3	arr[2]
5024	4.4	arr[3]
5032	5.5	arr[4]

Tabla comparativa: operación vs. número de movimientos necesarios para un arreglo de n=10

Operación	Fórmula de movimientos	Valor para n=10 (ejemplos)
Acceso arr[i]	0	0
Insertar al final (append) p = 10	$n - p = 0$	0
Insertar en el medio p = 5	$n - p = 10 - 5$	5 movimientos
Insertar al inicio p = 0	$n - p = 10 - 0$	10 movimientos
Insertar en p = 9 (antes del último)	$n - p = 10 - 9$	1 movimiento
Eliminar al final p = 9	$n - p - 1 = 10 - 9 - 1 = 0$	0 movimientos
Eliminar en el medio p = 5	$n - p - 1 = 10 - 5 - 1 = 4$	4 movimientos

Eliminar al inicio $p = 0$	$n - p - 1 = 10 - 0 - 1 = 9$	9 movimientos
Insertar sin capacidad	$\approx n$ (copiar todo)	≈ 10 movimientos (copias)

Responde: ¿Por qué insertar al inicio es más costoso que insertar al final?

Insertar al inicio obliga copiar todos los elementos existentes, mientras que insertar al final siempre que haya capacidad, se reduce a escribir en la siguiente celda.

2 Detective del Caso MegaStore

Fórmula matemática del costo de n inserciones con incremento de +1

Fórmula = $n(n-1)/2$

Fórmula del costo con duplicación de capacidad

$(n) = Co \cdot (2^m - 1)$ donde $m = \text{ceil}(\log_2(n / Co))$.

Informe breve (150 palabras) explicando a un gerente no técnico por qué el sistema era lento

La razón principal por la cual el sistema es lento es por su estructura, Cada vez que la estructura se queda sin capacidad, se asigna un nuevo arreglo de tamaño $2 \times \text{capacidad}$ y se copian los elementos existentes. Esto en eficiencia es mediocre, lo que ralentiza en gran medida, por eso a gran escala es muy poco viable seguir este método de duplicado.

Checkpoint Metacognitivo - Fase COMPRENDE

¿Puedo explicar con mis propias palabras por qué $\text{arr}[i]$ es $O(1)$ usando la fórmula de dirección?

Es por su estructura, $\text{arr}[i]$ es la posición en la que se encuentra, $O(1)$ es una constante, esto permite al arreglo acceder a cualquier posición.

¿Qué imagen mental tengo ahora de un arreglo en memoria? ¿Es diferente a lo que pensaba antes?

Si, un arreglo de memoria en esencia es más complejo de lo que imaginaba, pero es más simple de diferenciar, viendo desde la lógica puede ser en forma convencional un arreglo de una dimensión.

Si me preguntaran "¿por qué insertar al inicio es $O(n)$?", ¿podría dibujarlo paso a paso? No realmente, pero si entiendo el porque, en una lista de

datos desordenados se va a buscar en forma de una en una cada elemento necesario, por eso es que se inicia en $O(n)$

¿Qué conexión veo entre el análisis de complejidad de la Semana 1 y las operaciones de arreglos? Las complejidades y cómo se enfocan en diferentes puntos pero aun así están relacionados por los arreglos, en su caso, diferentes formas de anidamiento.

3 Arquitecto de Código

Código completo del TAD ArregloDinámico (pseudocódigo o lenguaje real)

Clase ArregloDinamico:

atributos:

```
datos[]    // arreglo interno
tamaño     // número de elementos almacenados
capacidad  // tamaño total del arreglo interno
capacidadMin // capacidad mínima permitida
```

método constructor():

```
capacidadMin ← 1
capacidad ← capacidadMin
tamaño ← 0
datos ← arreglo de tamaño capacidad
```

método agregar(elemento):

```
si tamaño == capacidad entonces
    redimensionar(capacidad * 2)
fin si
```

```
datos[tamaño] ← elemento
tamaño ← tamaño + 1
```

fin método

método insertar(índice, elemento):

```
si índice < 0 o índice > tamaño entonces
    lanzar Error("Índice fuera de rango")
fin si
```

```
si tamaño == capacidad entonces
    redimensionar(capacidad * 2)
fin si
```

```
para i desde tamaño - 1 hasta índice paso -1 hacer
    datos[i + 1] ← datos[i]
fin para
```

```

    datos[índice] ← elemento
    tamaño ← tamaño + 1
fin método

método eliminar(índice):
    si tamaño == 0 entonces
        lanzar Error("Arreglo vacío")
    fin si

    si índice < 0 o índice ≥ tamaño entonces
        lanzar Error("Índice fuera de rango")
    fin si

    para i desde índice hasta tamaño - 2 hacer
        datos[i] ← datos[i + 1]
    fin para

    tamaño ← tamaño - 1

    si tamaño > 0 y tamaño ≤ capacidad / 4 y capacidad / 2 ≥ capacidadMin entonces
        redimensionar(capacidad / 2)
    fin si
fin método

método obtener(índice):
    si índice < 0 o índice ≥ tamaño entonces
        lanzar Error("Índice fuera de rango")
    fin si

    devolver datos[índice]
fin método

método redimensionar(nuevaCapacidad):
    nuevoDatos ← arreglo de tamaño nuevaCapacidad

    para i desde 0 hasta tamaño - 1 hacer
        nuevoDatos[i] ← datos[i]
    fin para

    datos ← nuevoDatos
    capacidad ← nuevaCapacidad
fin método

fin clase

```

Tabla de complejidades: cada método con su O() temporal y espacial

Método	Complejidad Temporal	Complejidad Espacial
constructor	$O(1)$	$O(1)$
agregar(elemento)	$O(1) / O(n)$	$O(1)$
insertar(índice, elem)	$O(n)$	$O(1)$
eliminar(índice)	$O(n)$	$O(1)$
obtener(índice)	$O(1)$	$O(1)$
redimensionar	$O(n)$	$O(n)$

Lista de al menos 3 casos borde que tu implementación maneja

- **Agregar cuando el arreglo está lleno**
Se redimensiona antes de insertarla.
- **Insertar en índice 0 (inicio)**
Todos los elementos se desplazan correctamente a la derecha.
- **Insertar en índice = tamaño (final)**
Se comporta como agregar(elemento).

4 Laboratorio de Problemas Clásicos

Pseudocódigo de tu solución para cada problema

1. Rotar arreglo a la derecha en k posiciones

método rotarDerecha(A, k):

```

n ← tamaño de A
si n == 0 o n == 1 entonces
    retornar
fin si

```

```

k ← k mod n

```

```

reversa(A, 0, n-1)
reversa(A, 0, k-1)
reversa(A, k, n-1)

```

fin método

2. Eliminar duplicados de un arreglo ordenado (in-place)

método eliminarDuplicados(A):

```

n ← tamaño de A
si n == 0 entonces
    retornar 0
fin si

pos ← 1

para i desde 1 hasta n - 1 hacer
    si A[i] ≠ A[pos - 1] entonces
        A[pos] ← A[i]
        pos ← pos + 1
    fin si
fin para

retornar pos // nueva longitud

```

fin método

3. Mover ceros al final (manteniendo el orden)

método moverCeros(A):

```

n ← tamaño de A
pos ← 0

para i desde 0 hasta n - 1 hacer
    si A[i] ≠ 0 entonces
        A[pos] ← A[i]
        pos ← pos + 1
    fin si
fin para

mientras pos < n hacer
    A[pos] ← 0
    pos ← pos + 1
fin mientras

```

fin método

4. Evolución de la solución (mejora)

método elementoMayoritario(A):

```

candidato ← null
contador ← 0

para cada elemento x en A hacer
    si contador == 0 entonces

```

```

    candidato ← x
    contador ← 1
    sino si x == candidato entonces
        contador ← contador + 1
    sino
        contador ← contador - 1
    fin si
fin para

retornar candidato

fin método

```

Análisis de complejidad temporal y espacial de cada solución

1. Rotar arreglo a la derecha en k posiciones

Temporal: $O(n)$

(cada reversa recorre el arreglo)

Espacial: $O(1)$

(in-place, solo variables auxiliares)

2. Eliminar duplicados de un arreglo ordenado (in-place)

Temporal: $O(n)$

Espacial: $O(1)$

3. Mover ceros al final (manteniendo el orden)

Temporal: $O(n)$

Espacial: $O(1)$

4. Evolución de la solución (mejora)

Temporal: $O(n)$

Espacial: $O(1)$

Para al menos un problema, muestra la evolución de tu solución si la mejoraste

Mover ceros al final (in-place)

Idea mejorada

Usar dos punteros:

- **i** recorre el arreglo
- **pos** indica dónde colocar el siguiente elemento no cero

Primero copiamos todos los no-ceros al inicio y luego rellenamos el resto con ceros.

Checkpoint Metacognitivo - Fase APLICA

¿Puedo implementar `agregar()` sin mirar mis notas? ¿Qué parte me cuesta más recordar? no, aun necesito mirar las notas, y otra parte que necesito mejorar es el orden de los pasos y centrar bien la idea principal.

Cuando la IA me hizo preguntas socráticas, ¿en qué momento me di cuenta de algo que no había considerado? cuando tuve que verificar los movimientos de los arreglos, que eran de derecha a izquierda, me había basado en capacidad y en validar, no tome en cuenta la complejidad o el como se debía mover

De los 4 problemas, ¿cuál me costó más trabajo? ¿Qué patrón o técnica me faltaba? El problema 4, me faltó identificar si era un arreglo como también su secuencia

¿Cómo cambió mi aproximación a los problemas entre el primero y el último?

En el problema 1 revise que pasos seguir, y seguir procedimientos.

En el problema 4 tuve que verificar qué era lo que se debía conservar, el porqué y cómo funcionaba.

5 Diálogo Socrático sobre Decisiones de Diseño

Documento de reflexión con: cada decisión de diseño, la justificación refinada después del diálogo, y un escenario donde la cambiarías

1. Usé capacidad inicial de 10

Decisión

La estructura comienza con una capacidad inicial de 10 elementos.

Justificación refinada

Elegí 10 por intuición y porque suele ser suficiente para arreglos pequeños. En muchos casos no se usan todos los espacios, pero el desperdicio es pequeño y aceptable. Si el arreglo crece más de lo esperado, el costo se paga después al redimensionar. Esta decisión asume que el uso típico será reducido y que no pasa nada si se redimensiona varias veces.

Cuándo la cambiaría

La cambiaría si el arreglo se usa en un sistema con memoria muy limitada o si supiera de antemano que casi siempre se almacenarán muchos elementos. En ese caso, elegiría una capacidad inicial más ajustada al uso real.

2. Factor de crecimiento de 2x

Decisión

Cuando el arreglo se llena, su capacidad se duplica.

Justificación refinada

Duplicar reduce la cantidad de redimensionamientos, lo que mejora el rendimiento general. Aunque se desperdicia memoria temporalmente, el costo se paga pocas

veces. Elegí esta opción porque prioriza simplicidad y menos operaciones de copia, aun sabiendo que no es la más eficiente en uso de memoria.

Cuándo la cambiaría

La cambiaría en un contexto donde la memoria es más importante que la velocidad, usando por ejemplo un factor de 1.5x o incluso incrementos fijos.

3. No implementé reducción de capacidad

Decisión

El arreglo nunca reduce su tamaño interno aunque se eliminen muchos elementos.

Justificación refinada

Esto evita redimensionamientos frecuentes cuando el arreglo crece y decrece varias veces. Mantiene el código más simple y evita costos adicionales. Se asume que la memoria extra no es crítica en el contexto actual.

Cuándo la cambiaría

La cambiaría si el arreglo se usa en un sistema con memoria muy limitada o si se espera que el tamaño varía mucho a lo largo del tiempo.

4. Lanzo excepción si el índice está fuera de rango

Decisión

Si se accede a un índice inválido, se lanza una excepción con información clara.

Justificación refinada

Prefiero fallar de forma explícita y clara en lugar de continuar con datos incorrectos o comportamientos silenciosos. El mensaje ayuda a detectar errores rápidamente y no implica un costo significativo de memoria o rendimiento en este contexto

Cuándo la cambiaría

La cambiaría en sistemas donde no se permiten excepciones (por ejemplo, sistemas embebidos), usando códigos de error o validaciones externas.

5. Copio elementos uno por uno en el redimensionamiento

Decisión

Durante el redimensionamiento, los elementos se copian usando un bucle simple.

Justificación refinada

El bucle es fácil de entender, implementar y depurar. Para arreglos pequeños o medianos, el costo es aceptable. Aunque existen métodos más rápidos, la simplicidad es prioritaria en esta implementación.

Cuándo la cambiaría

La cambiaría en arreglos muy grandes o sistemas de alto rendimiento, usando copias por bloques o funciones optimizadas a bajo nivel.

Matriz de contexto vs. configuración ideal (al menos 4 contextos diferentes)

Contexto	Capacidad Inicial	Crecimiento	Reducción	Manejo de Errores	Copiado
----------	-------------------	-------------	-----------	-------------------	---------

Proyecto académico	10	2x	No	Excepciones	Bucle Simple
Sistema embebido (512 KB RAM)	Muy pequeña	1.2x-1.5x	si	Códigos de error	Copia Controlada
Aplicación de escritorio	10-50	1.5x-2x	Opcional	Excepciones	Bucle o Bloques
Servidor con mucha RAM	Grande	2x o más	No	Excepciones	Copia Optimizada

Una decisión que cambiarías de tu implementación original y por qué

Cambiaría el factor de crecimiento 2x por uno de 1.5x si el objetivo principal fuera reducir el desperdicio de memoria. Aunque implicaría más redimensionamientos, el uso de memoria sería más eficiente y equilibrado para aplicaciones reales.

6 Identificación de Patrones en Problemas de Arreglos

"Tarjetas de referencia" para cada patrón con: nombre, señales de detección, idea general

1. Dos Punteros

Señales de detección

“in-place”

“sin usar memoria extra”

“retorna la nueva longitud”

“mueve / filtra elementos”

Idea general

Dos índices con roles distintos: uno lee, otro escribe; o uno rápido y otro lento.

Recorres una sola vez, sin duplicar memoria.

2. Ventana Deslizante

Señales de detección

“subarreglo contiguo”

“longitud máxima / mínima”

“suma / promedio / conteo”

“k elementos”

Idea general

Mantener un rango activo que se mueve.

Actualizar el resultado incrementalmente al deslizar la ventana.

3. In-Place Modification

Señales de detección

“modifica el arreglo original”

“sin usar memoria adicional”

“retorna la nueva longitud”

Idea general

Reutilizar el mismo arreglo, sobrescribiendo valores válidos.

Solo importa que los primeros k elementos estén correctos; lo demás es irrelevante.

4. Prefijo / Sufijo

Señales de detección

“suma de un rango”

“producto de todos excepto el actual”

“muchas consultas”

“acumulado”

Idea general

Preprocesar acumulados (prefijo o sufijo) para responder rápido.

Cambia cálculos repetidos $O(n^2) \rightarrow O(n)$ o $O(1)$.

Para cada problema del Reto 4, indica qué patrón(es) usaste o podrías usar

Problema del Reto 4	Patrón usado / aplicable
Eliminar duplicados en arreglo ordenado	Dos Punteros + In-Place Modification
Mover ceros al final	Dos Punteros + In-Place Modification
Suma máxima de subarreglo de tamaño k	Ventana Deslizante
Producto de todos excepto el actual	Prefijo / Sufijo

Lista de 2-3 palabras clave que te harán pensar en cada patrón

Dos Punteros

“en el mismo arreglo”

“quitar o mover”

“reordenar”

Ventana Deslizante

“bloque seguido”

“pegados uno tras otro”

“máximo / mínimo”

In-Place Modification

“cambia el arreglo”

“sin memoria extra”

“nueva cantidad”

Prefijo / Sufijo

“sumado hasta aquí”

“desde... hasta...”

“multiplicar o sumar rápido”

Checkpoint Metacognitivo - Fase REFLEXIONA

¿Qué decisión de diseño me costó más justificar? ¿Qué dice eso sobre mi comprensión?

Me costo mas justificar 2 punteros porque es parecido a ventana deslizante, y me parecieron similares

¿Qué patrón de los explorados era nuevo para mí? ¿Puedo ahora reconocerlo en un problema?

El patrón prefijo/sufijo, parecía una suma acumulada pero terminó siendo más complicada

Si tuviera que enseñar "arreglos dinámicos" a un compañero, ¿por dónde empezaría?

Empezaré explicando la diferencia entre un arreglo dinámico y uno fijo.

¿Qué conexión inesperada descubrí entre los conceptos de esta semana?

Pude notar la relación que hay entre los patrones, tales como In-Place Modification y Dos Punteros, ya que uno lee y otro escribe dentro del mismo arreglo, como también la relación de Ventana Deslizante y Prefijo/Sufijo donde estos son dos formas de calcular rangos, uno en tiempo real y otro en el procesamiento.

7 Cazador de Bugs

Para cada implementación: bug identificado, línea específica, corrección propuesta

Implementación 1 – Bug en agregar()

Bug identificado: No se actualiza la variable capacidad después de redimensionar.

Línea específica:

datos = nuevoArreglo

// falta: capacidad = nuevaCapacidad

Corrección propuesta:

pseudocode

datos = nuevoArreglo

capacidad = nuevaCapacidad

Implementación 2 – Bug en insertar(índice, elemento)

Bug identificado: El bucle de desplazamiento pisa valores y accede mal a índices.

Línea específica:

para i desde tamaño hasta índice paso -1:

datos[i] = datos[i - 1]

Corrección propuesta:

para i desde tamaño hasta índice+1 paso -1:

datos[i] = datos[i - 1]

Implementación 3 – Bug en eliminar(índice)

Bug identificado: El bucle copia fuera de rango al eliminar el último elemento.

Línea específica:

para i desde índice hasta tamaño - 1:

datos[i] = datos[i + 1] // acceso inválido cuando i = tamaño - 1

Corrección propuesta:

pseudocode

para i desde índice hasta tamaño - 2:

datos[i] = datos[i + 1]

Implementación 4 – Bug en el constructor

Bug identificado: El arreglo interno se inicializa con longitud tamaño en lugar de capacidad.

Línea específica:

datos = nuevo arreglo de longitud tamaño // debería ser capacidad

Corrección propuesta:

datos = nuevo arreglo de longitud capacidad

Caso de prueba mínimo que revelaría cada bug

Implementación 1

Caso de prueba mínimo:

arr = nuevo ArregloDinámico(2)

arr.agregar(10)

```
arr.agregar(20)
arr.agregar(30) // falla aquí
```

Implementación 2

Caso de prueba mínimo:

```
arr = nuevo ArregloDinámico(3)
arr.agregar("A")
arr.agregar("B")
arr.agregar("C")
arr.insertar(1, "X") // debería quedar [A, X, B, C]
```

Implementación 3

Caso de prueba mínimo:

```
arr = nuevo ArregloDinámico(3)
arr.agregar("A")
arr.agregar("B")
arr.agregar("C")
arr.eliminar(2) // falla al eliminar último
```

Implementación 4

Caso de prueba mínimo:

```
arr = nuevo ArregloDinámico(3)
arr.agregar("A") // falla inmediatamente
```

Ranking de los bugs de más fácil a más difícil de detectar, con justificación

Implementación 4 (constructor) – Más difícil

Falla desde el primer uso.
No pasa ninguna prueba básica.
Muy evidente en desarrollo.

Implementación 3 (eliminar)

Falla solo al eliminar el último elemento.
Moderadamente fácil de detectar si se prueban casos extremos.

Implementación 2 (insertar)

Falla al insertar en medio o al inicio.
Puede pasar desapercibido si solo se prueba insertar al final.
Más difícil que el bug de eliminar porque corrompe datos silenciosamente.

Implementación 1 (agregar) – Más fácil

Funciona bien con pocos elementos.
El bug aparece solo al superar la capacidad inicial.
En producción puede tardar en manifestarse y causar errores intermitentes.